SIMATS ENGINEERING

ASSESSMENT TOOL 2

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3 Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _A JASWANTH REDDY (192412513)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Constraint-Based Problem Solving

1. Backtracking Search: Doctor–Shift Assignment

Given: Doctors D1, D2 and D3 must be assigned to Morning, Afternoon and Night. Each doctor receives exactly one shift and each shift has exactly one doctor.

Constraints

- D1 cannot work the Night shift.
- D2 must work before D3, so D2's shift must occur earlier than D3's shift.
- Only one doctor can be assigned to each shift.
- D3 cannot work the Morning shift.
- All three doctors must be assigned exactly one shift.

Backtracking procedure and constraint checks

Represent the variables as D1, D2 and D3, with domain {Morning, Afternoon, Night}. Assign a shift to one doctor at a time. After each assignment, check all constraints that can already be evaluated. If a constraint is violated, undo that assignment (backtrack) and try the next value.

1. Step 1: Assign D1 = Morning. Check: D1 is not assigned Night, so this is valid. Morning is now occupied.
2. Step 2: Try D2 = Afternoon. Check: the shift is unoccupied, so the assignment is valid so far.
3. Step 3: Try D3 = Night. Check: D3 is not assigned Morning; Night is unoccupied; and D2 (Afternoon) is before D3 (Night). All constraints are satisfied.
4. Step 4: Check completion. Every doctor has exactly one shift and every shift has exactly one doctor. No backtracking is needed for this branch.

Final valid schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Conclusion: The assignment D1–Morning, D2–Afternoon, D3–Night satisfies all stated constraints.

2. Grid Search: BFS and Manhattan Distance

Important note: The question states that obstacles block paths but does not show their exact positions or specify the coordinates of S and G. Therefore, the following explicit 5×5 example is assumed to demonstrate the method. A different supplied grid would produce a different expansion order and path.

Assumed grid coordinates: rows and columns are numbered 0–4. Start S = (0,0), Goal G = (4,4). Obstacles are (1,1), (1,2), (2,1) and (3,3). Movement is allowed only up, down, left and right; each move costs 1.

S	·	·	·	·

·	#	#	·	·
·	#	·	·	·
·	·	·	#	·
·	·	·	·	G

Legend: S = start, G = goal, # = obstacle, · = free cell.

Algorithm and heuristic

Breadth-First Search (BFS) uses a FIFO queue. It expands the earliest discovered unexpanded node first and marks cells as visited when they are enqueued, preventing repeated visits. Since every move costs 1, BFS finds a path with the minimum number of moves.

Manhattan distance is $h(r,c) = |r - 4| + |c - 4|$ for this assumed goal (4,4). It estimates the minimum number of horizontal and vertical moves to the goal when there are no obstacles. Standard BFS does not use h to prioritize nodes; it can be recorded as requested, but ordering the queue by h would change the algorithm. A* uses g+h for priority ordering.

Step-by-step BFS node expansion and queue updates

Neighbor order used for reproducibility: Right, Down, Left, Up. Queue entries below are shown as coordinates. The queue is FIFO; already visited cells and obstacles are skipped.

5. Start: enqueue (0,0). Queue = [(0,0)]. h(0,0)=8.
6. Expand (0,0): discover (0,1), (1,0). Queue = [(0,1), (1,0)].
7. Expand (0,1): (0,2) is new; (1,1) is an obstacle. Queue = [(1,0), (0,2)].
8. Expand (1,0): discover (2,0); (1,1) is blocked. Queue = [(0,2), (2,0)].
9. Expand (0,2): discover (0,3); (1,2) is blocked. Queue = [(2,0), (0,3)].
10. Expand (2,0): discover (3,0); (2,1) is blocked. Queue = [(0,3), (3,0)].
11. Expand (0,3): discover (0,4), (1,3). Queue = [(3,0), (0,4), (1,3)].
12. Expand (3,0): discover (4,0), (3,1). Queue = [(0,4), (1,3), (4,0), (3,1)].
13. Expand (0,4): discover (1,4). Queue = [(1,3), (4,0), (3,1), (1,4)].
14. Expand (1,3): discover (2,3); (1,4) is already visited. Queue = [(4,0), (3,1), (1,4), (2,3)].
15. Expand (4,0): discover (4,1). Queue = [(3,1), (1,4), (2,3), (4,1)].
16. Expand (3,1): discover (3,2), (2,1 is blocked). Queue = [(1,4), (2,3), (4,1), (3,2)].
17. Expand (1,4): discover (2,4). Queue = [(2,3), (4,1), (3,2), (2,4)].
18. Expand (2,3): discover (2,2); (3,3) is blocked. Queue = [(4,1), (3,2), (2,4), (2,2)].
19. Expand (4,1): discover (4,2). Queue = [(3,2), (2,4), (2,2), (4,2)].
20. Expand (3,2): (3,3) is blocked; other valid neighbors are already visited. Queue = [(2,4), (2,2), (4,2)].
21. Expand (2,4): discover (3,4). Queue = [(2,2), (4,2), (3,4)].
22. Expand (2,2): no new cells are added. Queue = [(4,2), (3,4)].
23. Expand (4,2): discover (4,3). Queue = [(3,4), (4,3)].
24. Expand (3,4): discover goal (4,4). Queue = [(4,3), (4,4)]. The goal has been discovered; shortest-path parent links can be used to reconstruct the route.

Optimal path and cost

Using the recorded parent links, one shortest path is:

(0,0) → (0,1) → (0,2) → (0,3) → (0,4) → (1,4) → (2,4) → (3,4) → (4,4)

This route contains 8 moves, so total cost = 8 × 1 = 8. Manhattan distance from S to G is 8, and this path reaches that lower bound; therefore it is optimal for the assumed grid.

3. Autonomous Rescue Robot in a Partially Observable Building

a) Constraints and their effect on decision-making

- Four-direction movement: the robot can move only up, down, left or right. Diagonal shortcuts are not allowed, so each candidate action is checked against the adjacent cells.
- Obstacles: blocked cells are forbidden states. The robot must not plan a route through a known obstacle and must revise its route if a newly observed cell is blocked.
- Limited sensing: the robot observes only adjacent cells. It cannot assume that the entire building map is known at the beginning; unseen cells are treated as unknown until sensed.
- Movement and risk costs: every move costs 1. Entering a risky zone adds 2, so that step costs 3 in total. A shorter route can be worse if it passes through several risky cells.
- Cost minimization and safety: the robot compares cumulative route costs and avoids blocked cells. When costs are equal, it can prefer the route with fewer risky exposures or more confirmed safe cells.
- Dynamic knowledge / online search: after each move, new observations update the map. If a route is blocked or risky, the robot replans using the information currently available.

b) Solution approach: Online Search with Uniform Cost Search

Uniform Cost Search (UCS) is appropriate for choosing a least-cost route because steps can have different costs. The robot combines UCS with online sensing: it plans only over currently known traversable cells, moves to a selected adjacent cell, senses again, updates its map and replans as needed.

25. Initialize the robot's known map with its current location S. Mark unobserved cells as unknown rather than assuming they are safe.
26. Sense the four adjacent cells. Mark each as free, blocked or risky according to the sensor information.
27. For each known free neighbor, calculate step cost: 1 for a normal cell and 3 for entering a risky cell (base cost 1 plus risk penalty 2). Never add blocked cells to the frontier.
28. Run UCS over the currently known map. Store the lowest cumulative cost g(n) for each reached cell and expand the frontier cell with the smallest g(n) using a priority queue.
29. Select the first safe next step on the lowest-cost route toward G. Execute only that step, because the rest of the route may include unseen cells.
30. Observe adjacent cells again and update the internal map. If the next route is blocked or a cheaper route becomes known, rerun UCS from the current position.

31. Repeat sensing, planning and movement until G is reached. Report the path and sum all movement and risk costs.

Cost formula: Total path cost = number of moves + 2 × (number of risky-zone entries). For example, a route with 6 moves and 2 risky entries costs 6 + 2(2) = 10.

c) Application of AI concepts

Intelligent agent

The robot is an intelligent agent because it perceives the environment through sensors, maintains an internal representation of known cells, chooses actions using a search strategy and acts to reach the survivor.

Rationality

A rational action is the one expected to achieve the goal with the lowest known cost while avoiding known obstacles and unnecessary danger, given the information available at that moment. Rationality does not mean the robot knows hidden obstacles in advance; it must act on its current percepts and revise decisions when new evidence arrives.

Environment type

- Partially observable: only adjacent cells can be sensed.
- Sequential: each move affects the robot's position and future choices.
- Dynamic from the agent's perspective: the robot's knowledge changes as new cells are observed; the building itself need not physically change.
- Discrete: rooms/cells and actions are represented as distinct states and moves.
- Known action costs but initially incomplete map: movement rules and cost rules are specified, while obstacle locations may be unknown.

d) Why this approach is suitable

UCS is suitable because it minimizes cumulative cost when actions have different costs, including the extra penalty for risky zones. Ordinary BFS minimizes the number of moves only when all actions have equal cost, so it may select a route that is short but more expensive or dangerous. Online sensing and replanning address partial observability: the robot does not depend on a complete map and can adapt when it discovers obstacles. The combined approach therefore respects movement restrictions, avoids known blocked cells, accounts for risk and updates decisions as knowledge improves.

Overall conclusion

Backtracking finds a feasible doctor-to-shift assignment by checking constraints after each choice. BFS finds a shortest path in an unweighted grid, as demonstrated using an explicitly assumed example because the original question does not specify exact grid coordinates or obstacles. For the rescue robot, online sensing combined with Uniform Cost Search is more suitable because it handles incomplete knowledge and minimizes total movement-plus-risk cost.